

★ Get unlimited access to the best of Medium for less than \$1/week. [Become a member](#)



Deep Learning based medical image segmentation — Part 4— Data Setup



Diya S

Following ▾

6 min read · Jul 5, 2023



41





Envato Elements — Image by nd3000

Now that we have downloaded the Glioblastoma MRI dataset from TCIA, saved them in our Google Drive, and created the base folders required by nnUNet, we are ready to set up the dataset for pre-processing and training.

First, we need to randomly split the dataset into training and test cohorts. A few points around the train:test split —

- We will not have a separate validation cohort since nnUNet utilizes a 5-fold cross-validation technique.
- I have not analyzed the image data to for any specific clinical characteristics (such as tumor size or grade) to determine if a random train:test split is the right approach, or if there needs to be a specific classification of images for better representation. For now, we will just go with a random split. We can analyze our approach later to see if it was the right one. Also, I believe Glioblastomas are all Grade 4 gliomas.
- The authors of the original paper have only released 147 ground truth expert segmentation maps out of the full 630 patient cohort. We will limit ourselves to **this group of 147 patients** so we can compare our model generated segmentation with the expert segmentation.
- I have randomly chosen 20% of the 147 patient cohort to be my test set. If you run my code, your randomized test set and eventual results may be different from mine. If you want to exactly duplicate my results, I will provide the test patient IDs and you can use the same set instead of randomly generating them.

Alright, let's get into it. We need to setup four different sets of files in the right file name format, and move them to the right folders for nnUNet to process —

1. Training segmentation files
2. Test segmentation files — not used by nnUNet, just for better

organization

3. Training MRI scans

4. Test MRI scans

Preparation

- Create a new Colab notebook — *01_t501_glio_data_setup.ipynb*
- Mount your Google Drive

```
from google.colab import drive
drive.mount('/content/drive')
```

- Import necessary Python packages

```
import os
import random
```

Randomizing the training:test data cohorts

- The Ground Truth segmentation files will drive all our next steps. We have 147 files. Let's get the patient ID number from the file names, and put them in a list. Then we can randomly select 20% of the list to be our

test dataset, and 80% to be our training dataset. Based on the patient IDs selected, we will need to move the MRI scans and labels for those patients to the training or testing folders. Remember, if you run this code, your randomly selected test dataset will be different from mine. Also nnUNet will not use the ground truth segmentation (label) files in the test dataset for its processing.

```
# Read all the ground truth segmentation files into a list
segm_src_dir = '/content/drive/MyDrive/TCIA/UPENN-GBM/images_segm/'
segm_file_list = os.listdir(segm_src_dir)

# Shuffle the list so the order is randomized
random.shuffle(segm_file_list)

# Set up the training data ratio and split the list at that point
train_ratio = 0.8
elements = len(segm_file_list)
train_elements = int(elements * train_ratio)

# Create new list of segmentation files that will be used for training
# nnUNet does not need the segmentation files for the test dataset, but we will
train_segm_list = segm_file_list[:train_elements] # Training set - From 0 - 80%
test_segm_list = segm_file_list[train_elements:] # Test set - From 80 - 100%

# Check how many elements and if they are randomized
print (train_segm_list)
len(train_segm_list)
```

Training segmentation (label) files:

- Use Python to generate Unix/Linux commands to move the training segmentation files to the right folders. This is a quick and dirty way of

moving the right files to the right directories. You can look at the nnUNet code for doing this more elegantly using Python packages. nnUNet requires the segmentation files to be in a certain format — *patientID.nii.gz*. We will use Python print statement to generate the Unix file move commands.

```
tr_segms_target_dir = '/content/drive/MyDrive/TCIA/nnUNet/nnUNet_raw/Dataset501_

# Filename is like this - UPENN-GBM-00428_11_segms.nii.gz
for file in train_segms_list:
    tr_segms_file = file.split('_')[0] # returns UPENN-GBM-00428
    patient_id = tr_segms_file.split('-')[2] # returns 00428
    just_id = int(patient_id.split('.')[0]) # returns 428
    print ('!mv ' + segms_src_dir + file + ' ' + tr_segms_target_dir + str(just_id)
```

```
[5] for file in train_segms_list:
    tr_segms_file = file.split('_')[0]
    patient_id = tr_segms_file.split('-')[2]
    just_id = int(patient_id.split('.')[0]) # Get the patient ID # and remove leading zeroes, and discard the .nii.gz extension
    print ('!mv ' + segms_src_dir + file + ' ' + tr_segms_target_dir + str(just_id) + '.nii.gz')

!mv /content/drive/MyDrive/TCIA/UPENN-GBM/images_segms/UPENN-GBM-00428.nii.gz /content/drive/MyDrive/TCIA/nnUNet/nnUNet_raw/Dataset501_Glioblastoma/labelsTr/428.nii.gz
!mv /content/drive/MyDrive/TCIA/UPENN-GBM/images_segms/UPENN-GBM-00105.nii.gz /content/drive/MyDrive/TCIA/nnUNet/nnUNet_raw/Dataset501_Glioblastoma/labelsTr/105.nii.gz
!mv /content/drive/MyDrive/TCIA/UPENN-GBM/images_segms/UPENN-GBM-00376.nii.gz /content/drive/MyDrive/TCIA/nnUNet/nnUNet_raw/Dataset501_Glioblastoma/labelsTr/376.nii.gz
!mv /content/drive/MyDrive/TCIA/UPENN-GBM/images_segms/UPENN-GBM-00261.nii.gz /content/drive/MyDrive/TCIA/nnUNet/nnUNet_raw/Dataset501_Glioblastoma/labelsTr/261.nii.gz
!mv /content/drive/MyDrive/TCIA/UPENN-GBM/images_segms/UPENN-GBM-00096.nii.gz /content/drive/MyDrive/TCIA/nnUNet/nnUNet_raw/Dataset501_Glioblastoma/labelsTr/96.nii.gz
!mv /content/drive/MyDrive/TCIA/UPENN-GBM/images_segms/UPENN-GBM-00062.nii.gz /content/drive/MyDrive/TCIA/nnUNet/nnUNet_raw/Dataset501_Glioblastoma/labelsTr/62.nii.gz
!mv /content/drive/MyDrive/TCIA/UPENN-GBM/images_segms/UPENN-GBM-00141.nii.gz /content/drive/MyDrive/TCIA/nnUNet/nnUNet_raw/Dataset501_Glioblastoma/labelsTr/141.nii.gz
!mv /content/drive/MyDrive/TCIA/UPENN-GBM/images_segms/UPENN-GBM-00029.nii.gz /content/drive/MyDrive/TCIA/nnUNet/nnUNet_raw/Dataset501_Glioblastoma/labelsTr/29.nii.gz
!mv /content/drive/MyDrive/TCIA/UPENN-GBM/images_segms/UPENN-GBM-00197.nii.gz /content/drive/MyDrive/TCIA/nnUNet/nnUNet_raw/Dataset501_Glioblastoma/labelsTr/197.nii.gz
!mv /content/drive/MyDrive/TCIA/UPENN-GBM/images_segms/UPENN-GBM-00148.nii.gz /content/drive/MyDrive/TCIA/nnUNet/nnUNet_raw/Dataset501_Glioblastoma/labelsTr/148.nii.gz
!mv /content/drive/MyDrive/TCIA/UPENN-GBM/images_segms/UPENN-GBM-00083.nii.gz /content/drive/MyDrive/TCIA/nnUNet/nnUNet_raw/Dataset501_Glioblastoma/labelsTr/83.nii.gz
!mv /content/drive/MyDrive/TCIA/UPENN-GBM/images_segms/UPENN-GBM-00131.nii.gz /content/drive/MyDrive/TCIA/nnUNet/nnUNet_raw/Dataset501_Glioblastoma/labelsTr/131.nii.gz
!mv /content/drive/MyDrive/TCIA/UPENN-GBM/images_segms/UPENN-GBM-00118.nii.gz /content/drive/MyDrive/TCIA/nnUNet/nnUNet_raw/Dataset501_Glioblastoma/labelsTr/118.nii.gz
!mv /content/drive/MyDrive/TCIA/UPENN-GBM/images_segms/UPENN-GBM-00009.nii.gz /content/drive/MyDrive/TCIA/nnUNet/nnUNet_raw/Dataset501_Glioblastoma/labelsTr/9.nii.gz
!mv /content/drive/MyDrive/TCIA/UPENN-GBM/images_segms/UPENN-GBM-00266.nii.gz /content/drive/MyDrive/TCIA/nnUNet/nnUNet_raw/Dataset501_Glioblastoma/labelsTr/266.nii.gz
!mv /content/drive/MyDrive/TCIA/UPENN-GBM/images_segms/UPENN-GBM-00274.nii.gz /content/drive/MyDrive/TCIA/nnUNet/nnUNet_raw/Dataset501_Glioblastoma/labelsTr/274.nii.gz
!mv /content/drive/MyDrive/TCIA/UPENN-GBM/images_segms/UPENN-GBM-00021.nii.gz /content/drive/MyDrive/TCIA/nnUNet/nnUNet_raw/Dataset501_Glioblastoma/labelsTr/21.nii.gz
!mv /content/drive/MyDrive/TCIA/UPENN-GBM/images_segms/UPENN-GBM-00264.nii.gz /content/drive/MyDrive/TCIA/nnUNet/nnUNet_raw/Dataset501_Glioblastoma/labelsTr/264.nii.gz
!mv /content/drive/MyDrive/TCIA/UPENN-GBM/images_segms/UPENN-GBM-00259.nii.gz /content/drive/MyDrive/TCIA/nnUNet/nnUNet_raw/Dataset501_Glioblastoma/labelsTr/259.nii.gz
```

- Copy all the move commands and paste them in a new cell. Run the cell. All your training label files are now moved to the *labelsTr* folder.

Test segmentation files

We can do the same for the segmentation files (label files) belonging to the test cohort, although nnUNet will not use it. It's just a nice way of organizing all our files for later use.

```
# This is not required by nnU-Net, but I am setting it up to use later for our
ts_segm_target_dir = '/content/drive/MyDrive/TCIA/nnUNet/nnUNet_raw/Dataset501_

for file in test_segm_list:
    ts_segm_file = file.split('_')[0] # Split the file name at the first underscore
    patient_id = ts_segm_file.split('-')[2] # Remove the string UPENN-GBM and return
    just_id = int(patient_id) # Get the patient ID # and remove leading zeroes
    print ('!mv ' + segm_src_dir + file + ' ' + ts_segm_target_dir + just_id + '.
```

- Copy the move commands generated for the test segmentation files into a new cell and run them.

Training MRI scans

- Now that we have moved the segmentation files, let's focus on the training MRI scans. They need to be moved into the *imagesTr* folder. nnUNet requires MRI scan files to end in a sequence that looks like — **patientID_4-digit-MRI-sequence.nii.gz**. For example, the raw scans for patient #428, which look like this — *UPENN-GBM-00428_11_T1.nii.gz*. This will need to be converted to *428_0000.nii.gz*.
- Our mapping of the MRI sequence to nnUNet sequence is — **T1:0000**, **T1-GD:0001**, **T2:0002**, and **FLAIR:0003**. Now, let's find the MRI scans for

patients in our training cohort and move them to the right nnUNet folder.

```
# Set up source and target directories for training files
mri_img_src_dir = '/content/drive/MyDrive/TCIA/UPENN-GBM/images_structural/'
tr_img_target_dir = '/content/drive/MyDrive/TCIA/nnUNet/nnUNet_raw/Dataset501_G

# Loop through training patient list and generate the move commands
# Each patient has their own folder under "images_structural", that looks like
for patient in train_segm_list:
    patient_id = patient.split('.')[0] # We have the patient ID from the segmenta
    patient_dir = patient_id + '_11' # Add the "_11" and to get the MRI folder na
    patient_dir_path = mri_img_src_dir + patient_dir # Generate full path of the
    just_id = int(patient_id.split('-')[2]) # Get the patient ID # and remove lea

    print ('!mv ' + patient_dir_path + '/' + patient_dir + '_T1.nii.gz ' + tr_img
    print ('!mv ' + patient_dir_path + '/' + patient_dir + '_T1GD.nii.gz ' + tr_i
    print ('!mv ' + patient_dir_path + '/' + patient_dir + '_T2.nii.gz ' + tr_img
    print ('!mv ' + patient_dir_path + '/' + patient_dir + '_FLAIR.nii.gz ' + tr_
```

- Copy the move commands into a new cell and run them.

Test MRI Scans

Medium

🔍 Search

✍ Write



```
# Set up source and target directories for training files
mri_img_src_dir = '/content/drive/MyDrive/TCIA/UPENN-GBM/images_structural/'
ts_img_target_dir = '/content/drive/MyDrive/TCIA/nnUNet/nnUNet_raw/Dataset501_G

# Loop through training patient list and generate the move commands
# Each patient has their own folder under "images_structural", that looks like
```



```
for patient in test_seg_list:
    patient_id = patient.split('.')[0] # We have the patient ID from the segmenta
    patient_dir = patient_id + '_11' # Add the "_11" and to get the MRI folder na
    patient_dir_path = mri_img_src_dir + patient_dir # Generate full path of the
    just_id = int(patient_id.split('-')[2]) # Get the patient ID # and remove lea

    print ('!mv ' + patient_dir_path + '/' + patient_dir + '_T1.nii.gz ' + ts_img
    print ('!mv ' + patient_dir_path + '/' + patient_dir + '_T1GD.nii.gz ' + ts_i
    print ('!mv ' + patient_dir_path + '/' + patient_dir + '_T2.nii.gz ' + ts_img
    print ('!mv ' + patient_dir_path + '/' + patient_dir + '_FLAIR.nii.gz ' + ts_
```

- Copy the move commands into a new cell and run them.

Phew! Finally, we have setup the dataset in the right format, and moved it into the right folders. Now the fun begins. We can start running nnUNet to pre-process, train and predict tumor segmentation. More in the next article. Please follow along and let me know if you have any questions or face issues setting this up.

[Artificial Intelligence](#)[Python](#)[Glioblastoma](#)[Deep Learning](#)



Written by Diya S

92 followers · 14 following

Following ▾

I am a high school student on a journey to understand how disruptive technologies can transform and democratize healthcare in the coming decade
D.S

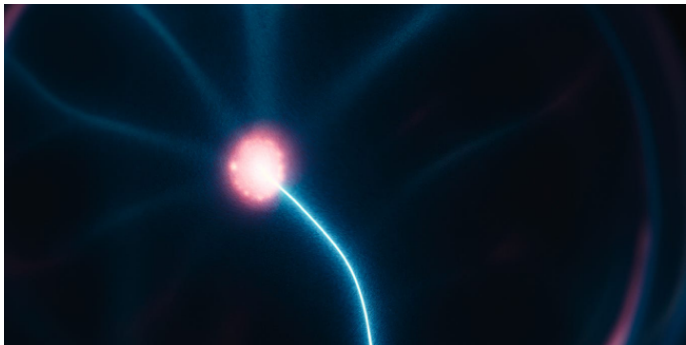
No responses yet



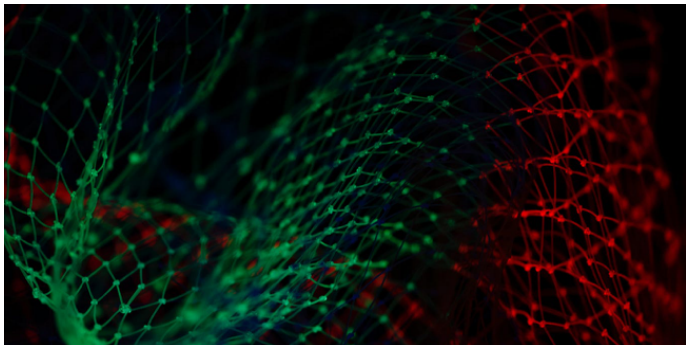
Charlie Chien

What are your thoughts?

More from Diya S



Diya S



Diya S

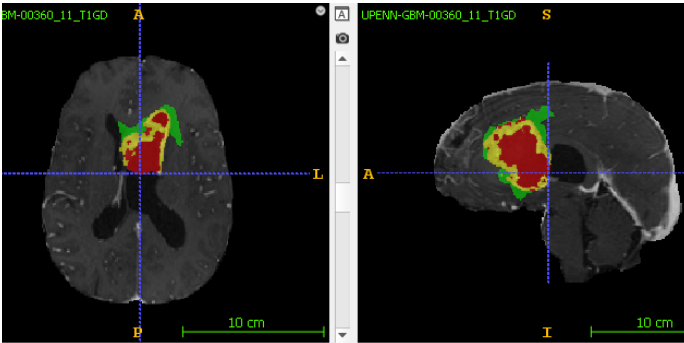
Glioblastoma brain tumor segmentation—Part 6—Neural...

Train a deep neural network model on Glioblastoma brain tumor multimodal MRI...

Jul 6, 2023

33

1



Diya S

Glioblastoma brain tumor segmentation—Part 1—Getting...

Start our automated brain tumor segmentation journey by downloading and...

May 14, 2023

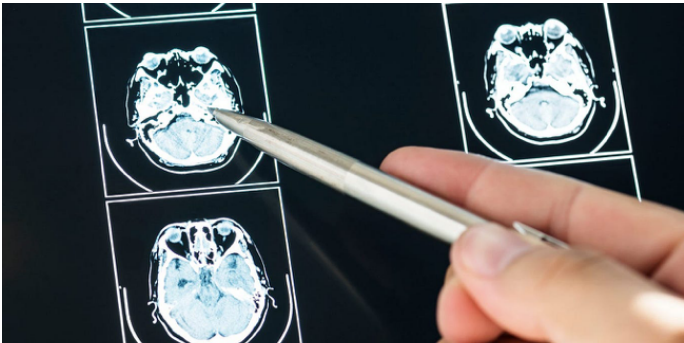
21

Glioblastoma brain tumor segmentation—Part 5—Data...

Generate image metadata, and preprocess images for standardization and intensity...

Jul 6, 2023

74



Diya S

Glioblastoma brain tumor segmentation—Part 3—Preparin...

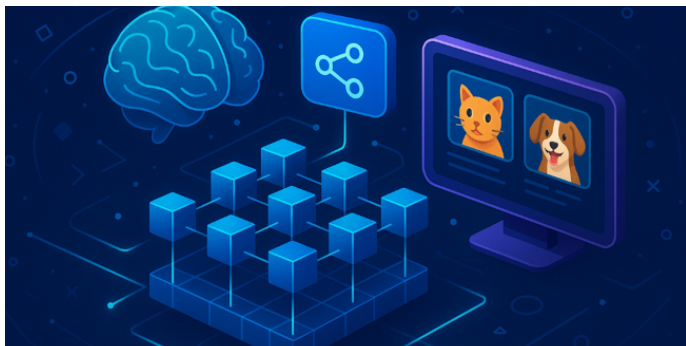
In our previous post, we downloaded the UPenn Glioblastoma MRI dataset from TCIA...

Jun 19, 2023

5

See all from Diya S

Recommended from Medium

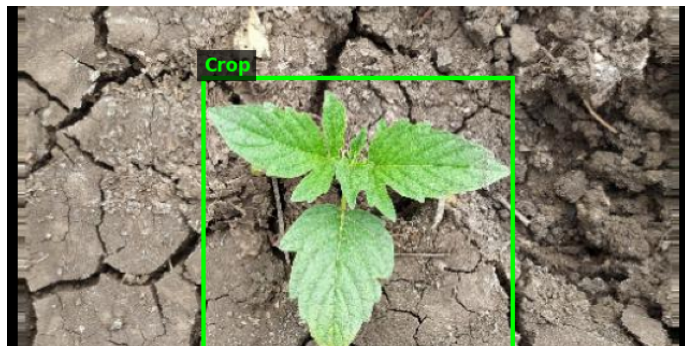


 Old Noisy Speaker

How AI Learn to See: Building a CNN in Python

Introduction: Teaching Machines to See

★ May 10

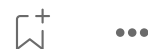


 Brian Hulela

Understanding Image Annotation Formats for Object Detection

A Beginner's Guide to YOLO, COCO, Pascal VOC, and More

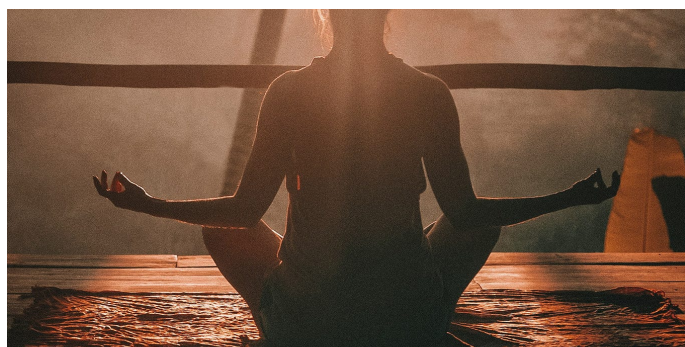
★ Apr 1



 Nikulsinh Rajput

This Pandas One-Liner Saved Me Hours of Data Cleaning

It was hiding in plain sight—and now I use it



 In Reclabs Inc by Odemakinde Elisha

Are 3D-CNN-LSTM Vision Encoders powerful for Video...

Have you ever wondered how GPTs today

it was hiding in plain sight — and now I use it in every project.

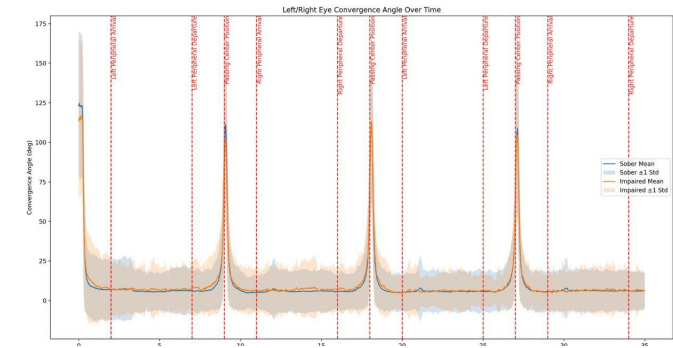
★ Jun 18 🖱 1

🔖 ⋮

Have you ever wondered how AI is today, handle video understanding? Here is a...

Mar 24 🖱 6

🔖 ⋮



 Siddhant Nitin

👁 Eye-Track-Analysis: Building an Impairment Detection System wi...

What if your eyes could speak—not just in expressions, but in data? In my latest side...

★ Apr 16 🖱 4

🔖 ⋮



 Toufiq Musah

Introduction to TorchIO for 3D MRI Processing: Augmentation...

In Part 1 of the Introduction to Torchio tutorial, we looked at how to perform...

May 11 🖱 70 💬 3

🔖 ⋮

See more recommendations